

TfL What-If Simulator

TfL Unified API Integration and Endpoint Usage

Purpose

This document describes how the Transport for London (TfL) Unified API was integrated into the London Underground Network Visualiser and What-If Simulator. It outlines the specific endpoints used throughout the project, the role each endpoint plays within the overall architecture, limitations encountered during development, and the rationale behind selecting particular data sources.

The TfL Unified API serves as the primary external data provider for the application. It supplies information relating to Underground lines, stations, disruptions and live train activity. This data is used to construct the network graph stored in Neo4j, support route calculation and provide real-time operational information within the frontend visualisation.

All API requests are made against:

<https://api.tfl.gov.uk>

Application credentials are stored securely using environment variables and are never hard-coded within the source code.

API Integration Architecture

The TfL API is utilised in two distinct ways within the project:

Static Network Construction

Certain endpoints are used during data ingestion to construct the Underground network graph. These endpoints define:

- Underground lines
- Station locations
- Route sequences
- Station connectivity

This data is processed and stored within Neo4j where it forms the foundation of the routing engine.

Live Operational Updates

Additional endpoints are queried periodically while the application is running in order to provide:

- Live service status information
- Station disruptions
- Live train visualisation

Unlike the graph construction data, this information changes dynamically and is not permanently stored within the database.

1. Underground Line Discovery

Endpoint

/Line/Mode/tube

Purpose

This endpoint returns all London Underground lines and their associated identifiers.

How It Is Used

The endpoint is queried during graph generation to obtain the complete set of Underground line IDs. These identifiers are then used to retrieve route sequence information for each line.

Without this endpoint, individual route sequence requests could not be generated automatically.

Data Retrieved

- Line name
- Line identifier
- Transport mode

Contribution to the Project

This endpoint forms the starting point of the entire network construction process.

Data retrieved from this endpoint is passed into the route sequence pipeline where station connectivity is generated.

2. Route Sequence Generation

Endpoint

/Line/{lineId}/Route/Sequence/all

Purpose

This endpoint provides the ordered sequence of stations served by a particular Underground line.

How It Is Used

For every line returned by /Line/Mode/tube, the application retrieves its route sequence.

The returned stop points are then transformed into:

Station Nodes

Each unique station becomes a Neo4j node.

Graph Relationships

Adjacent stations are converted into:

(:Station)-[:CONNECTS_TO]->(:Station)

relationships.

Additional Processing

The TfL API provides latitude and longitude coordinates for each station.

These coordinates are used to:

- calculate geographic distance
- support map rendering
- estimate train positioning
- support route visualisation

Contribution to the Project

This endpoint is responsible for creating the physical structure of the Underground network used by the routing engine.

Without it, neither route calculation nor map visualisation would be possible.

3. Live Line Status

Endpoint

/Line/Mode/tube/Status

Purpose

Provides current operational status information for Underground lines.

How It Is Used

The frontend periodically retrieves service information and displays it within the control panel.

Examples include:

- Good Service
- Minor Delays
- Severe Delays
- Part Suspended
- Suspended

Contribution to the Project

This allows users to compare real-world network conditions against hypothetical What-If scenarios.

It also demonstrates how external live data can be integrated into a graph-based simulation environment.

4. Detailed Service Status

Endpoint

/Line/Mode/tube/Status?detail=true

Purpose

Provides expanded disruption information beyond the standard status endpoint.

Additional Information Available

When disruptions occur, TfL may provide:

- affected route sections
- disruption descriptions
- directional information
- affected stations

How It Is Used

The application uses this information to determine whether lines should be treated as disrupted within live operating mode.

Contribution to the Project

This endpoint allows the system to move beyond simple status labels and provide a more realistic representation of current network conditions.

5. Station Disruptions

Endpoint

/StopPoint/Mode/tube/Disruption

Purpose

Provides disruption information affecting individual Underground stations.

How It Is Used

Used to identify:

- station closures
- restricted access
- lift outages
- partial operational restrictions

Contribution to the Project

This endpoint enables the simulator to reflect real station-level disruptions and prevents affected stations from being considered during route generation.

It also served as inspiration for the design of the What-If closure system.

6. Live Train Arrivals

Endpoint

/Line/{lineIds}/Arrivals

Purpose

Provides live train arrival predictions for Underground services.

How It Is Used

The application uses arrival predictions to estimate train locations and display live train movement on the map.

Data retrieved includes:

- line information
- destination data
- expected arrival times
- current station information

Challenges Encountered

One of the most significant findings during development was that train identifiers were not consistently reliable.

The original plan assumed that individual trains could be tracked across multiple API responses using a persistent identifier.

However:

- some train identifiers changed unexpectedly
- some trains disappeared entirely between updates
- some services returned incomplete information

This made true train tracking considerably more difficult than initially expected.

Solution

Fallback logic was introduced using:

- arrival predictions
- route progression estimates
- timing information

This produced smoother visual train movement and reduced the impact of inconsistent API responses.

TfL Data Sources Used Outside the Unified API

A significant discovery during development was that the TfL Unified API does not expose all information required for route optimisation.

Travel Time Data

The routing engine required realistic travel times between stations.

Initially it was assumed that these values would be available through the Unified API.

However, suitable travel-time information was not exposed through any practical endpoint.

Instead, travel times were extracted from publicly published TfL CSV datasets and processed into a format suitable for graph weighting.

These travel times were then attached to Neo4j relationships and used by Dijkstra's shortest-path algorithm.

This became a critical component of the routing engine.

Endpoints Evaluated but Not Used

Several TfL endpoints were investigated during development but were ultimately not incorporated.

Journey Planner API

TfL provides journey planning endpoints capable of generating complete routes.

These were intentionally not used.

Reason

The objective of the project was to implement and evaluate a custom graph-based routing system.

Using TfL's own routing engine would have bypassed one of the project's primary technical objectives.

The project therefore generates routes independently using Neo4j and Dijkstra's algorithm.

Crowding and Passenger Flow Data

Several datasets relating to passenger movement and station usage were explored.

Reason Not Implemented

While potentially valuable for future crowding simulations, incorporating this information would have substantially increased project complexity and exceeded the available development timeframe.

Key Lessons Learned

Several important lessons emerged from the API integration process:

- Real-world data is often less structured than expected.
- External APIs rarely provide all required information directly.
- Additional preprocessing and validation is usually necessary.
- Assumptions regarding identifier consistency should be verified early.
- Travel-time data can be more difficult to obtain than network topology data.
- Third-party APIs often require fallback strategies when data becomes incomplete or unavailable.

Perhaps the most important lesson was that successful integration involves far more than simply consuming an API. Significant effort was required to clean, transform and validate the data before it could be incorporated into the routing engine and visualisation system.

Conclusion

The TfL Unified API provided the foundation for both the structural and operational aspects of the London Underground Network Visualiser. Route sequence endpoints enabled construction of the underlying transport graph, while disruption and arrival endpoints supported live operational awareness and train visualisation.

Although several challenges were encountered, particularly relating to train tracking and travel-time availability, the API ultimately enabled the development of a realistic transport simulation environment that combines live Underground data with hypothetical disruption analysis through the What-If simulation framework.